

CSC241 Theory Assignment 1

Name: Laraib Kamran

Reg. No. sp26-bcs-028

Section: BCS

Instructor: Dr. Muhammad Shahid Bhatti

Assigned Condition: Resource limitation (last digit 8) room & equipment allocated Separately

Task 1: Stakeholder and Requirement Analysis

Stakeholders

- **Student:** want a booking process that feels fair. If every contested slot ends up going to a faculty member, students will feel the system is rigged even if the rule was reasonable.
- **Faculty members:** need their scheduled classes and genuinely urgent academic activity to not lose a room to an earlier, lower-priority booking.
- **Administrators:** want rooms used efficiently, and want to be able to see, after the fact, why utilization dropped in a given week.
- **Security staff:** need to know who was in a given room and when, in case something goes wrong.
- **Maintenance staff:** need to be able to pull a room out of service the moment it becomes unsafe, without waiting for anyone's sign-off.

Three Requirement Conflicts

1. **Fairness vs. priority.** The brief says students should have fair access to rooms, but faculty may sometimes need priority for teaching or urgent activities. It doesn't clearly explain what should happen if both need the same time slot. **Decision:** I assumed a fixed priority order that a confirmed teaching session comes first, next an urgent faculty request, and then an ordinary booking submitted later. This is my assumption because the brief does not specify a priority order.
2. **Utilization vs. safety.** Administration wants rooms to be used as much as possible, but maintenance staff may need to block a room immediately for safety reasons. This could affect someone who already has a booking. **Decision:** I assumed that a maintenance block takes priority over an existing confirmed reservation. I also assumed that utilization is calculated only using rooms that are actually available and not blocked, so maintenance issues do not unfairly lower the administration's utilization figures.
3. **Room availability vs. equipment availability (my assigned condition).** The brief mentions that some equipment is movable and limited in quantity. This means a room can be completely available while the projector cart needed for it is already being used somewhere else, and the opposite can also happen. **Decision:** I assumed that both the room and the required equipment must be available before a reservation can be confirmed. If the equipment is unavailable, the room hold is released instead of leaving the booking partially confirmed. The brief does not explicitly state this rule, so it is my design decision. This is also why Equipment is treated as a separate class in Task 2 rather than simply being a field inside Space.

Facts vs Assumptions

Facts stated in the brief: the university has not prescribed a design; a person may hold more than one role and roles can change during the semester; emergency events may cancel or relocate confirmed reservations and the system must keep enough information to audit this; some equipment is movable and scarce.

My assumptions (not stated in the brief): the exact priority order between teaching, urgent requests and ordinary bookings; that a maintenance block overrides a confirmed reservation immediately rather than waiting for the current booking to finish; that room and equipment are checked separately; that a cancelled reservation can never go back to confirmed, only be replaced by a brand-new one; and that audit log entries are never edited or deleted once written.

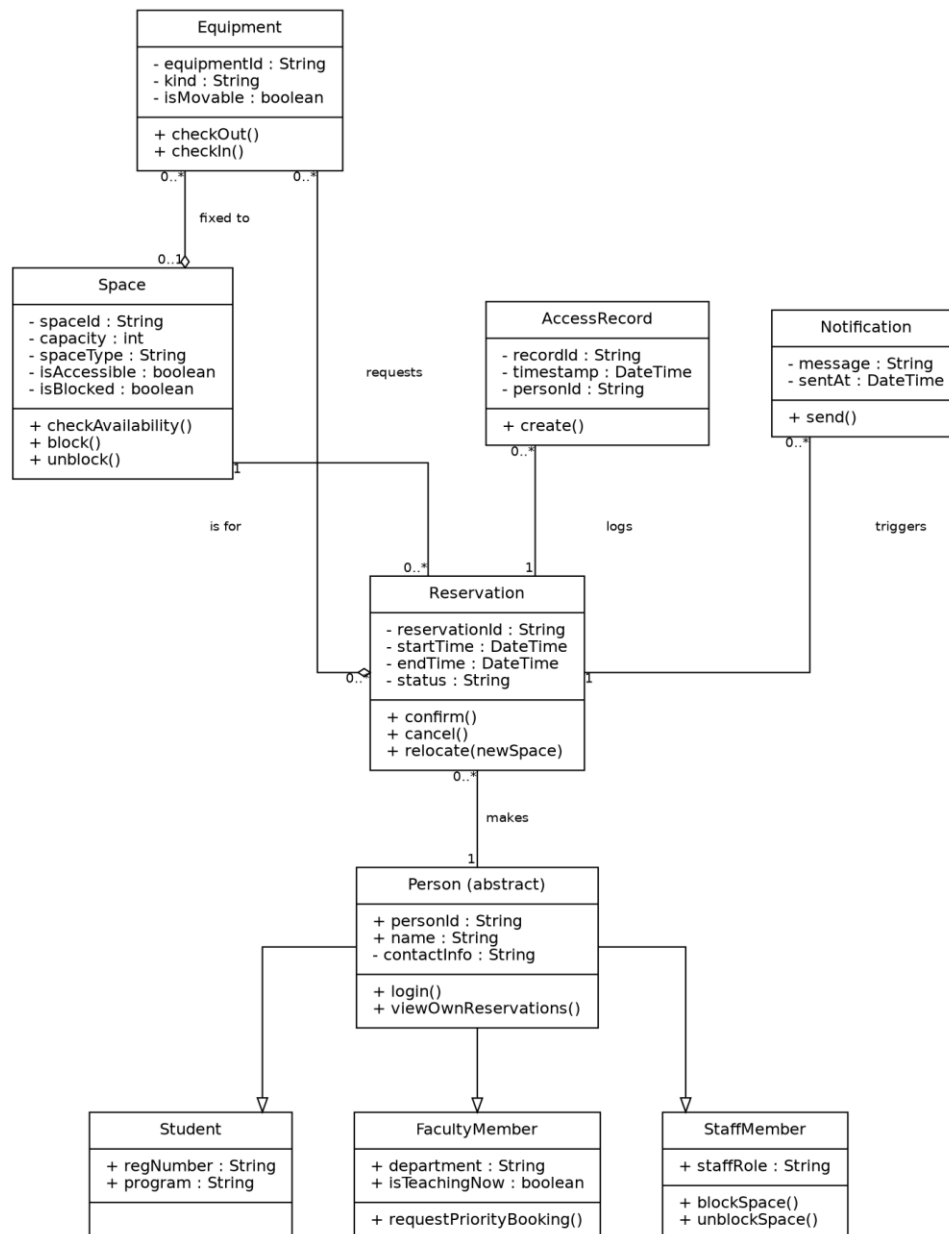
Task 2: Object Oriented Conceptual Model

My model has nine classes. I kept Space as a single class rather than four subclasses for classroom/lab/seminar/project space, because those four differ mainly in their capacity and equipment, not in how they behave i.e; a spaceType field does the same job as four nearly identical subclasses would, without the extra boilerplate.

Class	Responsibility	Key attributes / operations
Person (abstract)	Holds identity data every user shares, and lets someone log in and see their own bookings. Nothing role-specific lives here.	personId, name, contactInfo (hidden) / login(), viewOwnReservations()
Student	A person who can request a space like anyone else; adds student-only data.	regNumber, program
FacultyMember	A person who can additionally ask for a priority slot for teaching.	department, isTeachingNow / requestPriorityBooking()
StaffMember	Covers admin, security and maintenance staff; can block or unblock a Space.	staffRole / blockSpace(), unblockSpace()

Space	Represents one bookable room. Knows its own capacity, type and whether it is currently safe/free to use.	capacity, isAccessible, isBlocked / checkAvailability(), block()
Equipment	Represents one movable or fixed resource (projector, mic kit, lab kit) that a reservation may need.	kind, isMovable / checkOut(), checkIn()
Reservation	The actual booking. Owns its own life-cycle (requested → confirmed → cancelled/relocated/completed).	startTime, endTime, status / confirm(), cancel(), relocate()
AccessRecord	One log line proving a specific person accessed a specific space at a specific time.	timestamp, personId / create()
Notification	One outgoing message to a person, created whenever a reservation they care about changes.	message, sentAt / send()

UML Class Diagram



How responsibility is split

No single class runs the whole system. Space only knows about the room itself (capacity, whether it's blocked), it does not know who booked it. Reservation owns the booking life-cycle but does not manage the equipment inventory itself; it just asks Equipment whether an item is free. AccessRecord only records that an access happened; it does not decide whether access should be allowed, that decision belongs to Reservation and Space. Notification only knows how

to send a message; it does not decide when to send one. Splitting it this way means a change to, say, how blocking works only touches Space, not Reservation or Equipment.

Task 3: Abstraction, Encapsulation and Invariants

Three classes

Reservation abstracts the real-world act of one person booking one space for a time range. It leaves out everything that doesn't affect whether the booking is valid, it doesn't care what the meeting is about, only who, where, and when.

Space abstracts the booking relevant facts about a physical room: how many people fit, whether it has accessible features, and whether it's currently safe to use. It does not model the room's furniture, paint colour, or maintenance history in detail; none of that affects a booking decision.

Equipment abstracts a movable or fixed resource down to just what matters for allocation: what kind it is, whether it's currently checked out, and whether it can move between rooms at all.

Hidden data and controlled operations

Reservation: the status field should not be settable directly — it should only change through `confirm()`, `cancel()` and `relocate()`, so a status can never jump from Requested straight to Completed by mistake.

Space: `isBlocked` should only change through `block()/unblock()`, never by editing the field, otherwise a room could be silently marked safe again without anyone actually checking it.

Equipment: which reservation currently holds an item should only change through `checkOut()/checkIn()`, so two reservations can never both believe they have the same projector cart.

Invariants

1. A Space cannot have two confirmed Reservations with overlapping time ranges.
2. A Reservation cannot move to Confirmed while its Space is blocked.
3. One unit of Equipment cannot be checked out to two active Reservations at the same time.
4. Once a Reservation is Cancelled, it can never become Confirmed again — a new booking has to be made from scratch.
5. An AccessRecord, once created, is never edited or deleted, so the audit trail stays trustworthy.

Encapsulation is really what makes these rules enforceable. If any part of the program could set status or isBlocked directly, then any one of these invariants could be broken by a single careless line of code somewhere far from the class itself. Forcing every change through a method on the owning class means the invariant only has to be checked in one place.

Task 4: Alternative Designs and Decision

The scenario says a person can hold more than one role at once, and that roles can change mid-semester. Task 2 modelled this with inheritance (Student, FacultyMember, StaffMember all extend Person). The alternative is to give Person a list of Role objects (StudentRole, FacultyRole, SecurityRole, and so on) instead.

Criterion	Inheritance (Task 2 design)	Role objects (composition)
Holding multiple roles at once	Awkward: a person is exactly one subclass, so being a student and a TA at the same time needs an extra class or a workaround.	Natural: a Person just holds more than one Role object in a list.
Changing roles mid-semester	Hard: an object's class cannot change after it's created, so 'promoting' a student to TA means replacing the object.	Easy: add or remove a Role from the list; the Person object itself never changes.
Duplicate behaviour / coupling	Shared behaviour across roles has to live in Person, which can get crowded as more roles are added.	Each role's behaviour stays in its own small class, so classes stay focused.

For this scenario I would actually switch to role objects. The brief explicitly says roles change during the semester and that one person can hold more than one - that is exactly the situation inheritance handles badly. The Task 2 model uses inheritance because it is the more familiar starting point, but a refined version would replace Student/FacultyMember/StaffMember with a single User class that holds a list of Role objects, and priority rules like requestPriorityBooking() would belong to the FacultyRole object instead of a subclass.

Task 5: Behaviour and Edge Cases

Reservation life cycle (state transition table)

Current state	Trigger / event	Next state	Enforced by
---------------	-----------------	------------	-------------

Requested	Space and Equipment both pass availability check	Confirmed	Reservation, asking Space + Equipment
Requested	Space or Equipment already taken for that time	Rejected	Space / Equipment
Confirmed	User cancels before the start time	Cancelled	Reservation
Confirmed	Maintenance blocks the Space	Cancelled or Relocated	Space, then Reservation
Confirmed	Emergency closes the building	Relocated (or Cancelled if no space fits)	Reservation, driven by the emergency process
Confirmed	End time passes with no incident	Completed	Reservation
Relocated	End time passes at the new Space	Completed	Reservation

Three edge cases

Two simultaneous requests for the same space: whichever request's confirm() call reaches Space.checkAvailability() first gets Confirmed; the second is Rejected as soon as it checks. Space enforces this, not Reservation, because availability is a fact about the room, not about any one booking.

A maintenance block after confirmation: StaffMember calls Space.block(). Space itself does not touch the Reservation - it simply becomes unavailable. It is Reservation that notices its Space is now blocked and moves itself to Cancelled or Relocated, then creates a Notification. Splitting it this way means Space never has to know anything about bookings.

An emergency cancellation or relocation: an administrator (or an automatic emergency process) triggers this for every affected Reservation in a building. Each Reservation still enforces its own transition rules, it is only being told to cancel/relocate by an outside process, not having its status field edited directly and each one creates its own Notification and AccessRecord entry, so the audit trail covers the whole event.

Task 6: Independent Exploration: Immutable Value Objects

An immutable value object is an object whose state can never change after it is created, if you need different data, you make a new object instead of editing the old one (Makeyev, n.d.). The CMU SEI glossary defines an object as immutable when its state cannot be modified after construction and all its fields are effectively fixed once the constructor finishes (SEI CERT Oracle Coding Standard for Java, n.d.). The usual reason to do this is trust: once you have a reference to the object, you never have to worry that some other part of the program quietly changed it behind your back.

In my model, `AccessRecord` should be an immutable value object. Right now the class only has a `create()` operation and no way to modify a record afterwards, which matches this idea exactly — an audit entry that could be edited after the fact would defeat the whole point of an audit trail. Making this explicit (rather than just "a class with no setters") is useful because it tells anyone reading the design, in one word, that `AccessRecord` objects are safe to hand around and log without defensive copying, and that any bug report claiming a record was silently altered is really a bug, not a feature.

References

Makeyev, N. (n.d.). Value Objects — A Short Essay. Salesforce Engineering Blog.
<https://engineering.salesforce.com/value-objects-a-short-essay-db6036d0a77/>

SEI CERT Oracle Coding Standard for Java — Glossary (entry: "immutable"). Carnegie Mellon University Software Engineering Institute. <https://wiki.sei.cmu.edu/confluence/x/o1JGBQ>

Generative AI Disclosure

I used an AI assistant (Claude) to help organise this write-up and draft the class diagram from a design I worked through myself. I reviewed and edited the content, and I understand I am responsible for every claim and design decision in this submission.